VitalWatch

See the trend before the threshold.

TRACK
AI for Real-Time Decision Making

FORMAT
Hackathon idea proposal and technical plan

PROTOTYPE
FastAPI · HTML/CSS/JavaScript · ML · Historical hourly replay

A transparent, replayable early-warning research prototype that combines multivariate vital-sign trends, missingness signals, and an interpretable ML model to prioritize clinician assessment.

> *Safety boundary: alerts prioritize review; they do not diagnose sepsis or recommend treatment.*

2. Problem Statement

The core issue

Patient deterioration often develops as a pattern rather than a single dramatic measurement. Heart rate may gradually rise while respiratory rate increases, oxygen saturation declines, blood pressure weakens, and temperature changes. Each individual value may remain below a conventional alert threshold, while the combined trajectory is concerning.

A threshold-only workflow can therefore produce two problems:

- **Late recognition:** an alert appears only after a vital sign crosses a fixed limit.
- **Alert overload:** isolated threshold breaches generate noisy notifications without enough context.

VitalWatch addresses the gap by analyzing recent changes across several measurements and updating a patient-level risk score as new observations become available.

Target audience

The primary users are:

- Clinicians and nursing teams who need to prioritize patient review.
- Clinical informatics and quality-improvement teams exploring early-warning workflows.
- Researchers and datathon participants building transparent deterioration models.
- Health-tech teams evaluating how historical clinical data can support real-time-style decision tools.

The prototype is not intended to make autonomous clinical decisions. It surfaces a ranked worklist and supporting evidence for a qualified human reviewer.

Why now?

Clinical data is increasingly timestamped and available as a sequence of observations, but many demonstrations still focus on static dashboards or isolated threshold rules. There is an opportunity to combine:

- Causal rolling trends rather than future-aware summaries.
- Missingness indicators, because measurement frequency can reflect clinical workflow.
- Patient-level machine-learning evaluation rather than row-level random splits.
- A replayable interface that demonstrates how risk changes over time.

The PhysioNet/CinC Challenge 2019 dataset provides a practical starting point for a narrowly defined sepsis early-warning task. Its records are hourly rather than second-by-second bedside waveforms, so VitalWatch will replay them honestly as historical hourly observations.

3. Proposed Solution Overview

The pitch

VitalWatch is an AI-assisted clinical prioritization dashboard. It receives a historical or synthetic patient record one observation at a time, calculates recent multivariate trends, estimates the risk of a sepsis-related deterioration event within a defined future window, and presents the most concerning patients first with a concise evidence panel.

No IoT device is required for the prototype. A FastAPI replay service releases historical or synthetic rows chronologically so the dashboard behaves like a live system while preserving the data's original hourly resolution.

Key features

- **Patient priority board:** Rank patients by current model risk and risk change.
- **Chronological replay:** Play, pause, step forward, reset, and change playback speed.
- **Multivariate risk scoring:** Combine current values, rolling summaries, trends, and missingness indicators.
- **Threshold comparison:** Show how a simple threshold baseline compares with the learned model.
- **Risk history:** Plot the patient's risk score over time with alert and retrospective onset markers.
- **Vital-sign trends:** Display heart rate, respiratory rate, oxygen saturation, blood pressure, and temperature.
- **Missing-data visibility:** Show missing measurements explicitly rather than treating them as normal values.
- **Alert explanations:** Describe model-associated evidence such as rising respiratory rate or declining oxygen saturation.
- **Simulation labeling:** Clearly mark the interface as historical replay or synthetic demonstration mode.
- **Evaluation view:** Report lead time, sensitivity at a selected false-alert rate, and patient-level performance.

Value proposition

VitalWatch is valuable because it combines a credible ML workflow with a visible, replayable decision interface:

- It detects **patterns and trajectories**, not only isolated threshold breaches.
- It keeps the human decision-maker in the loop.
- It makes the model's current evidence visible rather than presenting an unexplained score.
- It demonstrates real-time-style behavior without requiring sensors, hospital integration, or protected live data.
- It is lightweight enough to build during a hackathon and extensible toward future streaming inputs.

Intended AI contribution

The AI component is a supervised early-warning model. At time `t`, it uses only observations available through `t` and predicts whether the patient will receive a sepsis label within a predefined future horizon, such as the next six hours.

The minimum model is an interpretable logistic regression. A fixed threshold rule serves as the baseline. The dashboard compares their alert timing and false-alert burden.

4. Architecture & Technical Methodology

Tech stack

Layer	Proposed technology	Purpose
Frontend	HTML, CSS, vanilla JavaScript	Custom, aesthetic clinical command-center interface
Charts	Apache ECharts or Chart.js	Responsive multi-series vital and risk charts
Backend	Python, FastAPI, Uvicorn	REST endpoints, replay control, prediction service
Live updates	WebSocket	Push each simulated observation and risk update to the browser
Data processing	Pandas, NumPy	Normalize records and create causal rolling features
ML	scikit-learn	Logistic regression, preprocessing, metrics, serialization
Storage	CSV/JSON for demo; SQLite or DuckDB for persistence	Synthetic data, predictions, replay state, evaluation results
Model artifact	Joblib	Save and load the trained prototype model
Deployment	Local server or container	Reproducible hackathon demonstration

FastAPI is preferred over Django for the MVP because the core workflow is an API, replay engine, and WebSocket stream. Django becomes attractive later if the system requires accounts, role-based permissions, an administrative console, audit logs, or multi-tenant workflows.

Data sources and data strategy

Model development and evaluation

Use the **PhysioNet/CinC Challenge 2019** dataset as the primary real-world starting point. It provides patient-hour records, vital signs, laboratory values, and a `SepsisLabel` outcome. The MVP can focus on:

- Heart rate (`HR`)
- Oxygen saturation (`O2Sat`)
- Temperature (`Temp`)
- Systolic blood pressure (`SBP`)
- Diastolic blood pressure (`DBP`)
- Respiratory rate (`Resp`)

Dashboard replay

Use a small synthetic CSV or JSON file containing stable, worsening, noisy, and missing-data patient trajectories. The synthetic rows are released chronologically by the replay engine. They are for interface demonstration and pipeline testing, not clinical validation.

Required row schema

```
patient_id
timestamp
heart_rate
respiratory_rate
oxygen_saturation
systolic_bp
diastolic_bp
temperature
heart_rate_missing
respiratory_rate_missing
oxygen_saturation_missing
blood_pressure_missing
temperature_missing
sepsis_label                # retrospective evaluation only
```

The model must not receive the future `sepsis_label` during replay. Blank values and explicit missingness indicators should be retained.

ML methodology

Target definition

For a prediction time `t`:

```
Input: all measurements available through time t
Target: whether the first sepsis label occurs within the next six hours
```

The prototype can define sepsis onset operationally as the first timestamp at which `SepsisLabel` equals 1. This is a dataset-based definition and must not be presented as the exact biological onset time.

Causal features

Features are calculated from the current and prior observations only:

- Current vital-sign values.
- Three-, six-, and twelve-hour rolling means, minima, maxima, and variability.
- Linear trend or slope for each vital sign.
- Change from a recent patient-specific baseline calculated using past data only.
- Hours since the last available measurement.
- Count of missing vital signs in the current window.
- Persistence of increasing or decreasing trends.

Models

1. **Threshold baseline:** A transparent rule that alerts when multiple vital signs cross prototype thresholds.
2. **Primary ML model:** Logistic regression with imputation, scaling, rolling features, and missingness indicators.
3. **Optional comparison:** A gradient-boosted tree model if time permits.

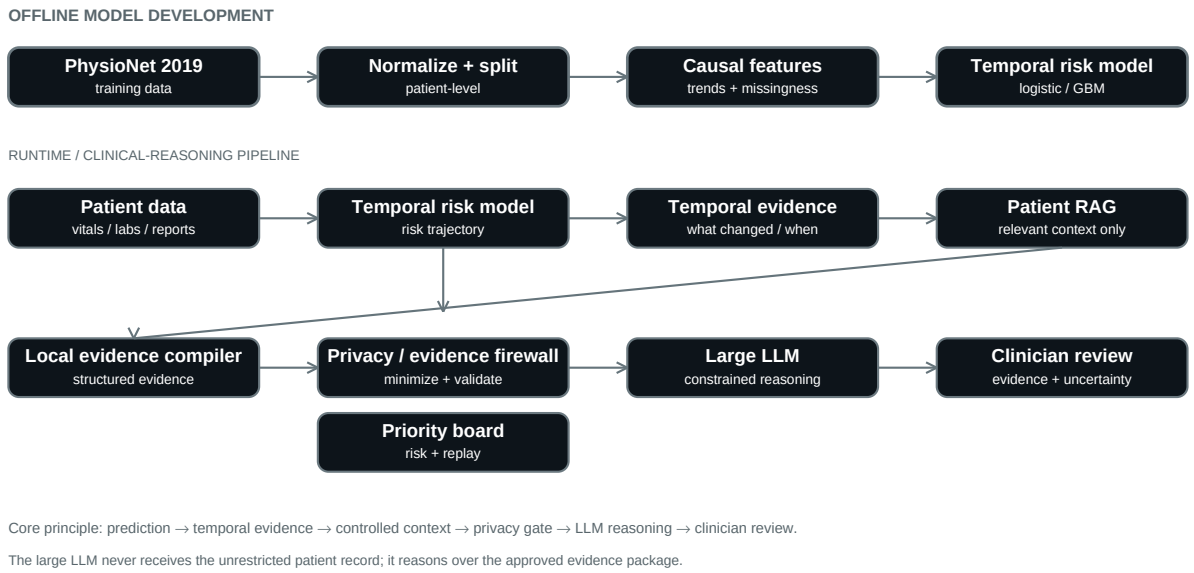
The first version should favor an interpretable model over a complex neural network. The project does not need an LLM or chatbot to qualify as AI; the learned sequential risk estimate is the core AI contribution.

Leakage controls

- Split data by patient, never by individual hourly rows.
- Keep all records from a patient in exactly one of train, validation, or test sets.
- Fit imputation, scaling, feature selection, and calibration on training data only.
- Calculate every rolling feature using only rows available at that prediction time.

- Do not use future labels, future vitals, or full-stay patient averages.
- Tune the alert threshold on validation patients and report final performance on held-out test patients.

System architecture / flow diagram



Core principle: prediction → temporal evidence → controlled context → privacy gate → LLM reasoning → clinician review.

The large LLM never receives the unrestricted patient record; it reasons over the approved evidence package.

Suggested API surface

```
GET  /                        Dashboard page
GET  /api/patients            Patient priority data
GET  /api/patients/{id}       Current patient state
GET  /api/patients/{id}/history Risk and vital history
POST /api/replay/start        Start playback
POST /api/replay/pause        Pause playback
POST /api/replay/step         Advance one simulated hour
POST /api/replay/reset        Reset replay state
WS   /ws/replay               Stream observations and predictions
```

Feasibility

The prototype is feasible within a hackathon because:

- The primary dataset and feature schema are well defined.
- The ML model can be implemented with standard scikit-learn components.
- The replay engine can use a CSV rather than requiring device integration.
- FastAPI provides REST and WebSocket support with little infrastructure.
- HTML/CSS/JavaScript allows a polished interface without adopting a large frontend framework.
- The first model can be trained offline and loaded by the API.
- The scope is intentionally narrow: sepsis-related early warning rather than general diagnosis.

5. Impact, Scalability & Roadmap

Impact metrics

The prototype should report both model performance and operational usefulness:

- **Lead time:** Time between the first sustained model alert and the retrospective labelled onset.
- **Sensitivity:** Percentage of sepsis-positive patients alerted before onset.
- **False alerts per patient-day:** Alert burden at a selected operating threshold.
- **Sensitivity at fixed false-alert rate:** Compare the ML model with the threshold baseline.
- **Patient-level AUROC and AUPRC:** Report performance on held-out patients.
- **Calibration:** Assess whether risk scores behave consistently as probability-like outputs.
- **Coverage of explanations:** Percentage of alerts with a readable evidence summary.
- **Replay latency:** Time from receiving a row to displaying the updated risk.

The primary demonstration should show one patient whose risk rises before the labelled onset while a stable patient remains low priority.

Scalability beyond the prototype

The architecture can evolve in stages:

1. Replace CSV replay with an authenticated EHR, HL7/FHIR, message-queue, or device gateway.
2. Replace in-memory replay state with durable storage and audit logging.
3. Add user authentication, role-based access, and organization-level data isolation.
4. Add model monitoring, drift detection, data-quality checks, and alert acknowledgement workflows.
5. Validate on external hospitals and different patient populations.
6. Add prospective silent-mode evaluation before any clinical workflow integration.
7. Support additional deterioration outcomes only after separate label and validation work.

MVP roadmap

Phase 1 — Project setup and data contract

- Confirm target definition and six-hour prediction horizon.
- Download and inspect the PhysioNet dataset.
- Normalize the six vital-sign fields and missingness indicators.
- Define the synthetic replay schema.

Phase 2 — Feature and model pipeline

- Build causal rolling features.
- Implement the threshold baseline.
- Train logistic regression with a patient-level split.
- Save the model and calculate validation metrics.

Phase 3 — Backend replay service

- Implement FastAPI endpoints.
- Load synthetic demo cases.

- Add step, pause, play, speed, and reset controls.
- Broadcast patient updates over WebSocket.

Phase 4 — Frontend dashboard

- Build the clinical command-center layout.
- Add patient priority cards and vital-sign cards.
- Add risk and vital-sign charts.
- Add alert markers, evidence text, and missing-data indicators.
- Add an always-visible simulation disclaimer.

Phase 5 — Evidence and final demo

- Replay one worsening and one stable patient.
- Compare model alerts with the threshold baseline.
- Calculate lead time and patient-level results.
- Prepare limitations and validation notes.
- Package the code, model artifact, synthetic data, and PDF proposal.

Suggested hackathon schedule

Time	Deliverable
Hours 0–4	Scope, data contract, repository setup, synthetic cases
Hours 4–10	Data normalization and causal feature pipeline
Hours 10–16	Baseline, logistic regression, patient-level evaluation
Hours 16–24	FastAPI endpoints and replay WebSocket
Hours 24–34	Frontend dashboard, charts, and visual polish
Hours 34–40	Connect model, tune replay, write explanations
Hours 40–48	Test, capture evidence, rehearse demo, finalize documentation

6. References & Appendix

Research and datasets

1. **PhysioNet/CinC Challenge 2019: Early Prediction of Sepsis from Clinical Data**

Dataset and challenge information: https://physionet.org/content/challenge-2019/1.0.0/

2. **PhysioNet**

Open research resources for physiological and clinical data: https://physionet.org/

3. **Reyna, M. A. et al.** Early Prediction of Sepsis from Clinical Data: The PhysioNet/Computing in Cardiology Challenge 2019. The challenge publication and citation information are available through the PhysioNet Challenge 2019 resource page.

Technical documentation

4. **FastAPI documentation**

https://fastapi.tiangolo.com/

5. **FastAPI WebSockets**

https://fastapi.tiangolo.com/advanced/websockets/

6. **scikit-learn documentation**

https://scikit-learn.org/stable/

7. **Apache ECharts documentation**

https://echarts.apache.org/en/index.html

Appendix A — Example synthetic patient trajectory

```
Patient: P001
Mode: Synthetic demonstration
Resolution: One observation per simulated hour

08:00  HR 82   RR 17  SpO2 98  BP 122/76  Temp 36.8  Risk 0.08
09:00  HR 85   RR 18  SpO2 97  BP 119/74  Temp 37.0  Risk 0.12
10:00  HR 89   RR 20  SpO2 96  BP 114/71  Temp 37.3  Risk 0.19
11:00  HR 94   RR 22  SpO2 95  BP 108/68  Temp 37.7  Risk 0.31
12:00  HR 99   RR 24  SpO2 94  BP 102/64  Temp --    Risk 0.44
13:00  HR 104  RR 26  SpO2 92  BP 96/60   Temp 38.4  Risk 0.67 ALERT
14:00  HR 108  RR 29  SpO2 91  BP 91/57   Temp 38.7  Risk 0.81 LABELLED ONSET
```

Appendix B — Demonstration language

Use the following wording in the dashboard and presentation:

> *“VitalWatch is a retrospective hourly-data replay demonstrating sequential risk updating. Alerts prioritize clinician assessment; they do not diagnose sepsis or recommend treatment.”*

> *“The explanation lists model-associated evidence, not proven clinical causes.”*

> *“A datathon model supports a research demonstration. Clinical deployment requires prospective, external, and workflow-specific validation.”*

Appendix C — Scope boundaries

The MVP deliberately does not claim:

- Second-by-second waveform monitoring.
- Continuous sensor integration.
- Clinical diagnosis.
- Treatment recommendations.
- Autonomous escalation.
- Generalization to all hospitals or patient groups.
- Prospective clinical benefit.

Appendix D — New AI Idea & Research Framing

The original temporal early-warning system remains the core predictive component. The new idea adds a controlled reasoning layer on top of it.

Updated one-line project statement

A system that combines temporal patient data with privacy-preserving LLM reasoning to detect deterioration and explain what might be happening.

What is new

- Temporal risk model: **Is the patient deteriorating?**
- Temporal evidence engine: **What changed, when, how fast, and how persistent?**
- Patient RAG: **What relevant historical context is needed?**
- Local evidence compiler: retrieved chunks → minimal structured evidence.
- Privacy / evidence firewall: validation, identifier minimization, leakage and prompt-injection checks.
- Large LLM: **What plausible explanations should a clinician consider, given this evidence?**

Correct framing

The LLM is not the primary detector and is not an autonomous diagnostic system. It is the **reasoning and explanation layer constrained by the temporal model**.

Research question

Does constraining an LLM with temporal physiological evidence and selectively retrieved patient context produce more reliable, explainable and privacy-conscious clinical reasoning than asking an LLM to reason directly from patient data?

This extends the original temporal deterioration idea rather than replacing it.

Appendix E — Privacy-Preserving Patient RAG

The RAG layer is not an unrestricted patient-memory system. Retrieval is conditioned by the deterioration event or reasoning task identified by the temporal system.

Stage	Role
Private patient data	Vitals, labs, reports, medications and prior history remain inside the controlled patient-data boundary.
Temporal evidence engine	Defines the current deterioration pattern and reasoning context.
Relevance-bounded Patient RAG	Retrieves only information relevant to the current event/task, with provenance.
Local evidence compiler	Converts raw retrieved chunks into strict structured evidence rather than forwarding raw text.
Privacy / evidence firewall	Identifier minimization, schema validation, leakage checks, prompt-injection checks and provenance.
Large LLM	Receives only the approved evidence package and performs constrained reasoning.
Clinician review	Shows differential possibilities, evidence, uncertainty and traceability.

Example

If the temporal system detects respiratory deterioration, retrieval may request recent respiratory reports, relevant medications and prior respiratory history. Unrelated history should not be retrieved simply because it exists.

Structured evidence passed to the LLM

Field	Example
Temporal event	Respiratory deterioration pattern
Observed change	SpO2 falling while respiratory rate rises
Persistence / baseline	Pattern persists and increasingly deviates from recent baseline
Relevant context	Only retrieved information necessary for interpretation
Contradictions / missing data	Evidence against an interpretation and unavailable context
Provenance	Where the item came from and why it was retrieved

Privacy evaluation

- Direct identifier extraction and indirect leakage tests.
- Prompt-injection tests on retrieved clinical notes.
- Minimum-disclosure comparison.
- Do not claim perfect anonymity; use privacy-preserving / privacy-conscious language.

Appendix F — Constrained LLM Reasoning & Updated Roadmap

The large LLM receives a structured evidence package instead of the raw patient chart. Its output is constrained so reasoning remains traceable and uncertainty is explicit.

LLM output contract

Output	Requirement
Differential possibilities	Plausible explanations, not confirmed diagnoses.
Supporting evidence	Major claims linked to supplied time-stamped evidence.
Evidence against	Contradictory or weakening evidence.
Temporal reasoning	Direction, persistence and timing.
Missing information	What would reduce uncertainty.
Uncertainty	Speculation clearly separated from evidence.
Evidence trace	Reviewer can inspect source evidence.

Experimental comparison

Snapshot LLM	Temporal LLM	Proposed system
Current measurements → LLM	Current + history → LLM	Temporal model → evidence → selective RAG → local compiler → firewall → LLM

Updated roadmap

- **Data/model:** retain PhysioNet 2019, patient-level splits, causal features and temporal ML.
- **Backend:** retain FastAPI replay/prediction service and WebSocket stream.
- **Frontend:** retain historical replay and priority board; add temporal-evidence view.
- **Reasoning:** add event-conditioned Patient RAG, local evidence compilation and provenance.
- **Privacy:** add evidence firewall and leakage / prompt-injection evaluation.
- **LLM:** add constrained differential reasoning with evidence, uncertainty and traceability.

Final safety framing

Alerts prioritize review. The LLM generates plausible differential insights from supplied evidence; it does not independently diagnose, prescribe treatment or escalate care. This remains a research prototype.

Architecture principle

Prediction → evidence → controlled context → privacy gate → reasoning → clinician review.

Appendix G - Future Development

SMARTPHONE-BASED ACTIVE HEALTH MONITORING | NOT PART OF VITALWATCH v1.0

Purpose. A later version of VitalWatch may accept selected, patient-initiated measurements captured with a smartphone. These observations would complement medical reports, not replace them. The current MVP remains the historical/synthetic early-warning replay and its temporal evidence, Patient RAG, privacy/evidence firewall and constrained LLM pipeline.

Planned smartphone capabilities

1. Camera-based resting heart-rate estimate

A guided fingertip-on-camera-and-flash session uses photoplethysmography (PPG) to estimate pulse rate. The app records the estimate, timestamp, device information and signal-quality result. Raw video should be processed on-device and discarded by default.

Output: Resting heart-rate estimate (BPM) plus quality metadata. This is not an ECG, blood-pressure or arrhythmia measurement.

2. Guided mobility assessment

A later app flow may guide a five-times sit-to-stand assessment. On-device video pose estimation or motion sensors can estimate repetition count and completion time, subject to a safe setup and appropriate screening.

Output: Completion time, repetition count and assessment-quality metadata. It is a functional measure, not a diagnosis.

How the future data would join the existing system

Guided phone session Patient starts a test	**On-device processing** Signal / pose + quality checks	**Structured observation** Value, unit, time, source, quality	**Existing VitalWatch pipeline** Temporal model → evidence → RAG → privacy gate → LLM

Integration principle: Each phone-derived result becomes a timestamped, source-labelled observation in the same general data contract used for other patient measurements. The existing temporal model and reasoning pipeline remain the core; a new input adapter supplies the additional observations.

Data, validation and privacy requirements

Data contract	Validation before any clinical use	Privacy safeguards
Observation type; value and unit; timestamp; source/device; signal or test quality; consent status; evidence ID. Preserve whether a value is measured, extracted, patient-reported or simulated.	Compare camera pulse estimates with an appropriate reference such as ECG; compare mobility counts/timing with trained-observer or reference measurements. Test repeatability, device variation, poor-signal rejection and usability in the intended population.	Obtain explicit consent; process video/audio locally where possible; avoid retaining raw media by default; minimize identifiers; encrypt stored data; apply role-based access and audit logging. Never send raw media to an external LLM by default.

Possible later expansion

After the smartphone workflow is independently tested, VitalWatch may add opt-in integrations with compatible wearables and external devices (for example, validated blood-pressure monitors or pulse oximeters). These sources should use the same timestamped observation structure and must retain device/source provenance. Device integration is a future extension, not a dependency of the current project.

Current MVP boundary: Smartphone sensing, wearables and external-device integration are explicitly excluded from the present implementation and primary experiments. VitalWatch v1.0 continues to use the defined clinical dataset, synthetic chronological replay, temporal risk model, temporal evidence, Patient RAG, privacy/evidence firewall and constrained LLM reasoning. Any future sensor-derived model use requires separate technical and clinical validation.